



MILESTONE 2 — DATA MODEL SPECIFICATION

# SupportPilot

MongoDB Collections for Knowledge Retrieval & Resolution — Weeks 3–4

|                      |                                                                                            |
|----------------------|--------------------------------------------------------------------------------------------|
| DOCUMENT TYPE        | Data model specification — M2 collections, document shapes, vector index, M1 relationships |
| PROJECT CODE         | SP-ATR-001 / M2-DATA                                                                       |
| COMPANION TO         | SupportPilot — Milestone 2 Execution Document                                              |
| BUILDS ON            | SupportPilot — Milestone 1 MongoDB Data Model (11 collections)                             |
| NEW COLLECTIONS      | 9                                                                                          |
| MODIFIED COLLECTIONS | 2 — both additive, <b>no migration required</b>                                            |
| KEY ADDITION         | Atlas Vector Search on article_chunks — no separate vector database needed                 |

# Table of Contents

---

**1. How M2 Attaches to M1**

What carries over, what changes, what is new

---

**2. Knowledge Base Collections**

Articles, versions, chunks, ingestion

---

**3. Resolution Collections**

Responses, citations, feedback

---

**4. Operational Collections**

KB gaps, retrieval logs

---

**5. Atlas Search Index Configuration**

Vector and text indexes

---

**6. Changes to M1 Collections**

Additive fields and the extended state machine

---

**7. Indexes**

New indexes and what they serve

---

**8. Transactions & Query Patterns**

Ingestion, retrieval, resolution

---

**9. Validation & Setup Checklist**

Schema validators and go-live checks

---

# 1. How M2 Attaches to M1

## 1.1 The 11 M1 collections

| M1 collection               | M2 status | What M2 does with it                                        |
|-----------------------------|-----------|-------------------------------------------------------------|
| users                       | Unchanged | Knowledge Manager role gains authoring permissions          |
| tickets                     | Extended  | Adds resolution state; status machine gains PENDING_REVIEW  |
| ticket_comments             | Unchanged | An accepted resolution is posted as a source: "AI" comment  |
| classification_overrides    | Unchanged | —                                                           |
| category_taxonomy           | Extended  | Adds kb_article_count so coverage gaps are visible in admin |
| sla_policies                | Unchanged | —                                                           |
| business_calendars          | Unchanged | —                                                           |
| departments / teams / sites | Unchanged | Article access filtering by department                      |
| applications                | Unchanged | affected_system becomes a retrieval filter                  |
| activity_logs               | Unchanged | New actions: article published, resolution sent             |
| counters                    | Unchanged | —                                                           |

## 1.2 Why no migration is required

M1's tickets collection embeds classification as an object and references everything unbounded. M2 adds two scalar flags to tickets and creates nine entirely new collections. Existing tickets remain valid; they simply have has\_resolution: false by absence, which is accurate.

**The one M1 decision that pays off here**

Keeping classification as a nested object rather than flat fields means the M3 multi-agent flow can add keys inside it without touching the schema again. The same principle applies to ticket\_responses.steps[] in this milestone — each step is an object so M3 can add validation verdicts per step.

## 1.3 The 9 new M2 collections

| Collection          | Type          | Purpose                                                       |
|---------------------|---------------|---------------------------------------------------------------|
| knowledge_articles  | Content       | The KB article itself, with publish workflow                  |
| article_versions    | Content       | Immutable version history                                     |
| article_chunks      | Index         | <b>Vector + text search target.</b> The retrieval collection. |
| ingestion_jobs      | Operational   | Bulk import with per-document failure tracking                |
| ticket_responses    | Transactional | Generated resolutions with embedded steps                     |
| response_citations  | Transactional | Resolved citations, queryable independently                   |
| resolution_feedback | Transactional | Helpful / not helpful from the requester                      |
| kb_gaps             | Operational   | Refusals aggregated into a Knowledge Manager work queue       |
| retrieval_logs      | Diagnostic    | Why a given article was retrieved. TTL 30 days.               |

## 2. Knowledge Base Collections

### knowledge\_articles

The authored article. Moderate size, edited by humans, read on every KB page.

```
{
  _id: ObjectId,
  slug: "vpn-connection-timeout-troubleshooting", // unique
  title: "VPN Connection Timeout – Troubleshooting Guide",

  category: "VPN", // → category_taxonomy (M1)
  sub_category: "Connection failure",
  tags: ["anyconnect", "timeout", "firewall", "remote access"],

  content: "## Overview\nUsers may see...", // markdown
  content_hash: "sha256:8f3a2c...", // skips re-embedding unchanged content

  status: "PUBLISHED", // DRAFT | PUBLISHED | ARCHIVED
  version: 3,

  // ---- provenance ----
  source_system: "CONFLUENCE", // MANUAL|CONFLUENCE|SHAREPOINT|UPLOAD
  source_url: "https://wiki.company.com/display/IT/VPN-Timeout",
  source_updated_at: ISODate("2026-06-14"),

  // ---- access control ----
  visible_to_departments: [], // empty = all
  is_internal_only: false, // true = hidden from the employee portal

  // ---- indexing state ----
  last_indexed_at: ISODate,
  indexed_version: 3, // < version means the index is stale
  chunk_count: 7,
  embedding_model: "embed-v1",
  index_error: null,

  author_id: ObjectId,
  author_name: "Meera R.",
  reviewed_by_id: ObjectId,
  created_at: ISODate,
  updated_at: ISODate
}
```

**On indexed\_version:** comparing it against version is how the weekly consistency sweep finds articles whose content changed but whose chunks were never rebuilt. Without it, a stale index is invisible.

**Indexes:** {slug:1} unique · {status:1, category:1} · {status:1, indexed\_version:1, version:1} for the sweep · {updated\_at:-1}

### article\_versions

Insert-only. Same immutability principle as quote versions in the freight project.

```
{
  _id: ObjectId,
  article_id: ObjectId,
  version: 2, // unique with article_id
  title: "VPN Connection Timeout – Troubleshooting Guide",
  content: "## Overview\n...",
  change_note: "Added firewall port 4500 step",
  changed_by_id: ObjectId,
  changed_by_name: "Meera R.",
  changed_at: ISODate
}
```

**Indexes:** {article\_id:1, version:-1} · unique {article\_id:1, version:1}

### article\_chunks

★ The retrieval collection. Carries the embedding vector and both search indexes.

```
{
  _id: ObjectId,
  article_id: ObjectId, // → knowledge_articles
  chunk_index: 3, // ordinal within the article

  // ---- denormalised from the parent – avoids $lookup on every search ----
  article_title: "VPN Connection Timeout – Troubleshooting Guide",
  article_status: "PUBLISHED", // filter field on the vector index
  article_slug: "vpn-connection-timeout-troubleshooting",
  category: "VPN", // filter field
  article_updated_at: ISODate, // lets the model flag stale guidance

  // ---- the chunk itself ----
  heading_path: "VPN Troubleshooting > Timeout Errors > Client Configuration",
  content: "Verify the VPN client server address is set to " +
```

```

        "corp-vpn.company.com. An incorrect address produces a " +
        "timeout rather than an authentication error...",
token_count: 612,

// ---- the vector ----
embedding:      [0.0213, -0.1184, 0.0871, ...], // 1024 floats
embedding_model: "embed-v1", // filter field - model change = re-index
embedding_dim:  1024,

created_at: ISODate
}

```

### Three fields are denormalised onto every chunk, and that is deliberate

article\_title, article\_status and category are duplicated from the parent. Atlas Vector Search filters must operate on fields inside the indexed collection

— you cannot filter a \$vectorSearch stage on a joined document. Without these, every search would retrieve archived articles and filter them out afterwards, wasting the candidate budget. The cost is that archiving an article requires updating its chunks; do that in the same transaction.

**Indexes:** kb\_vector\_index (section 5) · kb\_text\_index (section 5) · {article\_id:1, chunk\_index:1} · {embedding\_model:1} for re-index sweeps

## ingestion\_jobs

Per-document failure tracking. One bad PDF must not fail a 200-document batch.

```

{
  _id:      ObjectId,
  job_type: "BULK_UPLOAD", // BULK_UPLOAD|CONFLUENCE_SYNC|REINDEX
  source_ref: "kb-import/2026-08/it-runbooks.zip",
  triggered_by_id: ObjectId,

  status: "COMPLETED_WITH_ERRORS", // QUEUED|RUNNING|COMPLETED|
                                     // COMPLETED_WITH_ERRORS|FAILED

  progress: {
    total_documents: 214,
    processed:       214,
    articles_created: 198,
    articles_updated: 9,
    failed:          7,
    chunks_created:  1_642,
    chunks_embedded: 1_642
  },

  errors: [
    { document: "printer-setup-v2.pdf", stage: "LOAD",
      message: "Scanned PDF with no text layer - OCR required" },
    { document: "network-diagram.docx", stage: "CHUNK",
      message: "No heading structure; produced 1 chunk of 8400 tokens" },
    { document: "sso-config.md", stage: "EMBED",
      message: "Provider timeout after 3 retries" }
  ],

  started_at: ISODate,
  finished_at: ISODate,
  duration_ms: 184_320
}

```

**Indexes:** {status:1, started\_at:-1} · {triggered\_by\_id:1}

### 3. Resolution Collections

#### ticket\_responses

The generated resolution. Steps embedded — bounded, always read together.

```
{
  _id:      ObjectId,
  ticket_id: ObjectId,           // → tickets (M1)
  ticket_number: "IT-2026-004521", // denormalised for list views

  // ---- the answer ----
  sufficient_context: true,
  summary: "VPN timeouts on the corporate network are usually caused by " +
    "firewall rules blocking UDP 500/4500 or a stale cached credential.",

  steps: [                       // EMBEDDED – bounded, ordered
    { order: 1,
      instruction: "Verify the corporate firewall allows VPN traffic on " +
        "UDP ports 500 and 4500.",
      sources: [ { article_id: ObjectId, chunk_index: 3,
        article_title: "VPN Troubleshooting Guide",
        heading_path: "Timeout Errors > Firewall",
        rerank score: Decimal128("0.9120") } ],
      requires_approval: false,
      is_destructive: false },

    { order: 2,
      instruction: "Check the VPN client server address is corp-vpn.company.com.",
      sources: [ { article_id: ObjectId, chunk_index: 1, ... } ],
      requires_approval: false, is_destructive: false },

    { order: 3,
      instruction: "Clear cached credentials and restart the VPN service.",
      sources: [ { article_id: ObjectId, chunk_index: 4, ... } ],
      requires_approval: true,      // credential change → human approval
      is_destructive: true,
      destructive_reason: "Clears stored credentials" }
  ],

  escalation_recommended: false,
  escalation_reason:      null,

  // ---- guardrail audit ----
  steps_generated: 4,
  steps_dropped: 1,           // uncited → removed
  dropped_details: [
    { instruction: "Contact your network administrator to reset the tunnel.",
      reason: "NO_VALID_CITATION" }
  ],

  // ---- provenance ----
  confidence:      Decimal128("0.8700"),
  confidence_parts: { top_rerank: Decimal128("0.9120"),
    citation_coverage: Decimal128("1.0000"),
    classification: Decimal128("0.9200") },

  retrieval_log_id: ObjectId,      // → retrieval_logs
  queries_used: ["VPN connection timeout error",
    "AnyConnect cannot connect corporate network"],
  chunks_retrieved: 5,

  model:      "claude-x",
  prompt_version: "resolution.v1",
  embedding_model: "embed-v1",
  tokens_in: 4_120,
  tokens_out: 680,
  latency_ms: 2_410,

  // ---- lifecycle ----
  status: "DRAFT",                // DRAFT|SENT|EDITED_SENT|REJECTED
  reviewed_by_id: null,
  reviewed_at: null,
  edit_diff: null,                // set on EDITED_SENT – quality signal
  reject_reason: null,

  created_at: ISODate
}
```

#### Why steps\_dropped and dropped\_details are stored

The citation guardrail silently removes uncited steps before the agent ever sees them. Recording what was dropped, and why, is your only visibility into hallucination rate. If this number climbs after a prompt change, the change made things worse — and you would not otherwise know.

**Indexes:** {ticket\_id:1, created\_at:-1} · {status:1, created\_at:-1} · {sufficient\_context:1, created\_at:-1} · {steps\_dropped:1} for quality monitoring

## response\_citations

Flattened copy of what is embedded in steps. Exists for independent querying.

```
{
  _id:      ObjectId,
  response_id: ObjectId,
  ticket_id: ObjectId,
  step_order: 1,

  article_id: ObjectId,
  article_title: "VPN Troubleshooting Guide",
  article_slug: "vpn-connection-timeout-troubleshooting",
  chunk_index: 3,
  heading_path: "Timeout Errors > Firewall",
  snippet:      "Verify the corporate firewall allows VPN traffic on...",

  rerank_score: Decimal128("0.9120"),
  retrieval_rank: 1,
  created_at:   ISODate
}
```

**Why duplicate what is already embedded in steps[]?** Two different questions need answering. "Show me this resolution" reads the embedded copy in one document. "Which articles are actually being cited, and which have never been used?" needs an aggregation across all citations — which an embedded array cannot serve efficiently. The duplication is small and write-once.

**Indexes:** {response\_id:1} · {article\_id:1, created\_at:-1} — article usage analytics · {ticket\_id:1}

## resolution\_feedback

```
{
  _id:      ObjectId,
  response_id: ObjectId,
  ticket_id: ObjectId,
  user_id:   ObjectId,
  was_helpful: true,
  comment:   "Step 3 fixed it",
  resolved_ticket: true,           // did the ticket close without escalation
  created_at: ISODate
}
```

**Indexes:** {response\_id:1} · {was\_helpful:1, created\_at:-1}

## 4. Operational Collections

### kb\_gaps

Comes free from the refusal path. This is the Knowledge Manager's work queue.

```
{
  _id:      ObjectId,
  category: "PRINTER",
  sub_category: "Driver installation",

  occurrence_count: 23,           // incremented on each refusal
  ticket_ids: [ObjectId, ObjectId, ...], // capped at 50 most recent
  sample_queries: [
    "printer driver install windows 11",
    "cannot add network printer driver",
    "printer driver missing after update"
  ],
  sample_subjects: [
    "Cannot install printer driver on new laptop",
    "Network printer not detected after Windows update"
  ],

  status: "OPEN",                // OPEN|WRITING|RESOLVED|WONT_FIX
  assigned_to_id: null,
  resolved_by_article_id: null,

  first_seen: ISODate,
  last_seen:  ISODate
}
```

**Indexes:** {category:1, sub\_category:1} unique · {status:1, occurrence\_count:-1} — the queue, highest impact first

Upsert on {category, sub\_category} with \$inc on the counter and \$push with \$slice: -50 on the ticket array. One atomic operation per refusal.

### retrieval\_logs

Diagnostic. TTL 30 days. Looks like overhead until someone asks "why that article?"

```
{
  _id:      ObjectId,
  ticket_id: ObjectId,
  response_id: ObjectId,

  original_subject: "VPN connection failing on corporate network",
  queries_extracted: ["VPN connection timeout error", "..."],
  query_extraction_ms: 340,

  vector_hits: [                // top 30, trimmed to top 10 for storage
    { chunk_id: ObjectId, article_title: "VPN Troubleshooting Guide",
      chunk_index: 3, score: Decimal128("0.8412"), rank: 1 }
  ],
  keyword_hits: [
    { chunk_id: ObjectId, article_title: "Network Firewall Configuration",
      chunk_index: 0, score: Decimal128("6.2100"), rank: 1 }
  ],
  fused_top20: [
    { chunk_id: ObjectId, rrf_score: Decimal128("0.0308"), rank: 1 }
  ],
  reranked_top5: [
    { chunk_id: ObjectId, rerank_score: Decimal128("0.9120"), rank: 1 }
  ],

  latency: { query_ms: 340, vector_ms: 118, keyword_ms: 84,
    fusion_ms: 3, rerank_ms: 290, total_ms: 835 },
  degraded: [],                // e.g. ["RERANKER_UNAVAILABLE"]
  created_at: ISODate
}
```

**Indexes:** {ticket\_id:1} · {created\_at:1} TTL 30 days · {"latency.total\_ms":-1} for tail analysis

## 5. Atlas Search Index Configuration

### 5.1 Vector index

```
// Index name: kb_vector_index    ·    Collection: article_chunks
{
  "fields": [
    { "type": "vector",
      "path": "embedding",
      "numDimensions": 1024,
      "similarity": "cosine" },

    { "type": "filter", "path": "article_status" },
    { "type": "filter", "path": "category" },
    { "type": "filter", "path": "embedding_model" }
  ]
}
```

The three filter fields are exactly the denormalised fields from section 2. `embedding_model` as a filter means that during a model migration you can index new chunks alongside old ones and switch atomically by changing the filter value.

### 5.2 Text index

```
// Index name: kb_text_index      ·    Collection: article_chunks
{
  "mappings": {
    "dynamic": false,
    "fields": {
      "content":      { "type": "string", "analyzer": "lucene.standard" },
      "heading_path": { "type": "string", "analyzer": "lucene.standard" },
      "article_title": { "type": "string", "analyzer": "lucene.standard" },
      "article_status": { "type": "token" },
      "category":      { "type": "token" }
    }
  }
}
```

### 5.3 The two queries

```
// VECTOR half
[
  { $vectorSearch: {
    index: "kb_vector_index", path: "embedding",
    queryVector: [...], numCandidates: 300, limit: 30,
    filter: { article_status: "PUBLISHED", embedding_model: "embed-v1" }
  }},
  { $project: { content:1, article_id:1, article_title:1, chunk_index:1,
    heading_path:1, article_updated_at:1,
    score: { $meta: "vectorSearchScore" } }}
]

// KEYWORD half
[
  { $search: {
    index: "kb_text_index",
    compound: {
      filter: [ { text: { query: "PUBLISHED", path: "article_status" } } ],
      should: [
        { text: { query: q, path: "content" } },
        { text: { query: q, path: "heading_path",
          score: { boost: { value: 2 } } } },
        { text: { query: q, path: "article_title",
          score: { boost: { value: 3 } } } }
      ]
    }
  }},
  { $limit: 30 },
  { $project: { content:1, article_id:1, article_title:1, chunk_index:1,
    heading_path:1, score: { $meta: "searchScore" } }}
]
```

`numCandidates` should be roughly  $10 \times \text{limit}$ . Too low costs recall; too high costs latency with no gain.

#### Atlas Search indexes are eventually consistent

Newly inserted chunks are not immediately queryable — typically a few seconds. Ingestion verification must

#### poll

rather than query once and report failure. Integration tests need the same retry loop. This catches teams out on day one and produces phantom "embedding failed" bug reports.

## 6. Changes to M1 Collections

### 6.1 tickets

```
// ADDED in M2 – both scalar, both default-absent
has_resolution:      true,
resolution_status:   "DRAFT",    // DRAFT|SENT|EDITED_SENT|REJECTED|NONE
latest_response_id:  ObjectId,

// status enum EXTENDS – all M1 values remain valid
// M1: NEW|CLASSIFIED|AI_PROCESSING|ASSIGNED|IN_PROGRESS|
//     WAITING_ON_USER|ESCALATED|RESOLVED|REOPENED|CLOSED|FAILED|CANCELLED
// M2 adds: PENDING_REVIEW    (between AI_PROCESSING and ASSIGNED)
```

The new transition: AI\_PROCESSING → PENDING\_REVIEW when a resolution is generated and awaiting agent review, then PENDING\_REVIEW → IN\_PROGRESS on accept or → ESCALATED on reject. Add both to the state machine guard table.

**New index:** {resolution\_status:1, created\_at:-1}

### 6.2 category\_taxonomy

```
// ADDED in M2
kb_article_count:  6,
kb_coverage:       "GOOD",      // GOOD|THIN|MISSING – from the day-1 audit
last_gap_seen_at:  ISODate
```

Surfacing coverage next to each category in the admin UI makes the content gap visible to the people who can fix it, rather than buried in a spreadsheet from week 3.

## 7. Indexes

```
// == NEW COLLECTIONS ==
knowledge_articles { slug:1 } unique
                  { status:1, category:1 }
                  { status:1, indexed_version:1, version:1 } stale sweep
                  { updated_at:-1 }

article_versions  { article_id:1, version:-1 }
                  { article_id:1, version:1 } unique

article_chunks    kb_vector_index  (Atlas Vector Search)  *
                  kb_text_index   (Atlas Search)         *
                  { article_id:1, chunk_index:1 }
                  { embedding_model:1 }

ingestion_jobs    { status:1, started_at:-1 }
                  { triggered_by_id:1 }

ticket_responses  { ticket_id:1, created_at:-1 }          *
                  { status:1, created_at:-1 }
                  { sufficient_context:1, created_at:-1 }
                  { steps_dropped:1 }

response_citations { response_id:1 }
                  { article_id:1, created_at:-1 } usage analytics
                  { ticket_id:1 }

resolution_feedback { response_id:1 }
                   { was_helpful:1, created_at:-1 }

kb_gaps           { category:1, sub_category:1 } unique
                  { status:1, occurrence_count:-1 }          * queue

retrieval_logs    { ticket_id:1 }
                  { created_at:1 } TTL 30 days
                  { "latency.total_ms":-1 }

// == ADDED TO M1 ==
tickets           { resolution_status:1, created_at:-1 }
```

## 8. Transactions & Query Patterns

### 8.1 Multi-collection writes

| Operation          | Collections touched                                                                                                                                                                                       |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Publish an article | Update knowledge_articles status, insert article_versions, delete old article_chunks, insert new ones. <b>Transaction</b> — a published article with half its old chunks retrieves contradictory content. |
| Archive an article | Update the article <b>and</b> its chunks' article_status. <b>Transaction</b> — missing the chunks means archived content stays retrievable.                                                               |
| Store a resolution | Insert ticket_responses, bulk-insert response_citations, update tickets flags and status. <b>Transaction.</b>                                                                                             |
| Record a refusal   | Insert response with sufficient_context: false, upsert kb_gaps with \$inc. <b>Transaction.</b>                                                                                                            |
| Accept and send    | Update response status, insert a ticket_comments row with source: "AI", transition the ticket. <b>Transaction.</b>                                                                                        |
| Bulk ingestion     | Per-document transaction, not per-batch. One bad document must not roll back 200 good ones.                                                                                                               |

### 8.2 Query patterns

| Screen or job                 | Shape                                                                                                                                               |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Retrieval (the hot path)      | Two aggregations run <b>concurrently</b> (asyncio.gather), fused in Python. Do not run them sequentially — that doubles the latency for no benefit. |
| Ticket detail with resolution | One findOne on tickets, one on ticket_responses. Steps and their citations are embedded, so the whole panel renders from two documents.             |
| Citation click-through        | findOne on knowledge_articles by article_id, scroll to heading_path. Agent verifies in two seconds.                                                 |
| KB gap queue                  | find({status:"OPEN"}).sort({occurrence_count:-1})                                                                                                   |
| Article usage analytics       | aggregate on response_citations grouped by article_id. Reveals which articles are never cited — candidates for rewriting or retirement.             |
| Stale index sweep             | find({status:"PUBLISHED", \$expr:{\$lt:["\$indexed_version","\$version"]}}). Weekly job.                                                            |
| Self-help portal search       | Same hybrid pipeline, plus is_internal_only: false filter. Reuses the retrieval code entirely.                                                      |

## 9. Validation & Setup Checklist

### 9.1 Schema validator — the dimension guard

```
db.createCollection("article_chunks", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["article_id", "chunk_index", "content", "embedding",
              "embedding_model", "article_status"],
    properties: {
      chunk_index: { bsonType: "int", minimum: 0 },
      content:     { bsonType: "string", minLength: 20 },
      token_count: { bsonType: "int", minimum: 50, maximum: 1200 },
      article_status: { enum: ["DRAFT", "PUBLISHED", "ARCHIVED"] },
      embedding: { bsonType: "array",
                    minItems: 1024, maxItems: 1024,    // ← dimension guard
                    items: { bsonType: "double" } }
    }
  }},
  validationLevel: "strict", validationAction: "error"
})
```

The minItems/maxItems pair is the important line. A dimension mismatch otherwise fails **silently at query time** — the vector search simply returns nothing, and there is no error to trace.

### 9.2 Checklist

| <input type="checkbox"/> | Item                                                                                                              |
|--------------------------|-------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | All 9 new collections created with \$jsonSchema validators, validationAction: "error"                             |
| <input type="checkbox"/> | Embedding dimension enforced by minItems/maxItems, not by convention                                              |
| <input type="checkbox"/> | kb_vector_index and kb_text_index both created and confirmed queryable                                            |
| <input type="checkbox"/> | Ingestion verification polls for index availability rather than querying once                                     |
| <input type="checkbox"/> | Archiving an article updates its chunks in the same transaction — tested                                          |
| <input type="checkbox"/> | Publish rebuilds chunks atomically; no window where old and new chunks coexist                                    |
| <input type="checkbox"/> | Vector and keyword searches run concurrently, verified by latency measurement                                     |
| <input type="checkbox"/> | Uncited steps are dropped and recorded in steps_dropped / dropped_details                                         |
| <input type="checkbox"/> | Refusal writes a kb_gaps upsert in the same transaction as the response                                           |
| <input type="checkbox"/> | retrieval_logs TTL index confirmed working — check after 30 days or force with a short TTL in staging             |
| <input type="checkbox"/> | Portal projection test: no confidence, model version, internal notes or retrieval_log_id in any employee response |
| <input type="checkbox"/> | Stale index sweep job scheduled and tested against a deliberately stale article                                   |
| <input type="checkbox"/> | All indexes in section 7 verified with explain() showing IXSCAN                                                   |
| <input type="checkbox"/> | Classification and ticket creation still work with the AI service stopped                                         |